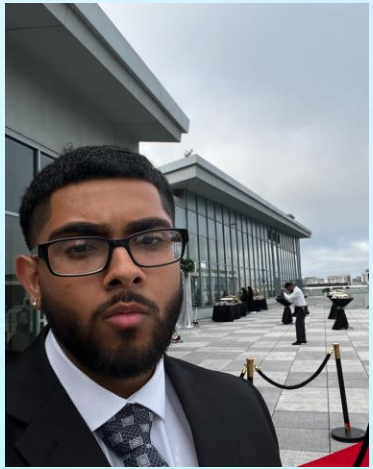
Stock Exchange Database

CSE 111 – Database System
Project Checkpoint 1

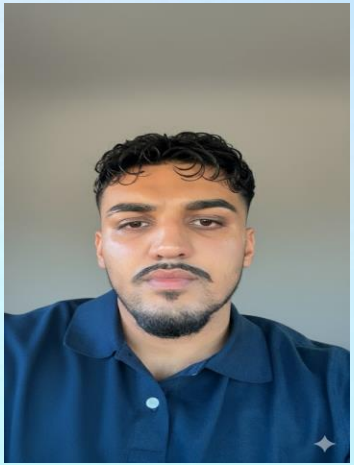
Ajay Grewal
Rohit Kumar
Arshdeep Dhaliwal

The Team

Ajay Grewal
Contact:
agrewal20@ucmerced.edu



Rohit Kumar
Contact:
rkumar43@ucmerced.edu



Arshdeep Dhaliwal
Contact:
adhaliwal25@ucmerced.edu

Project Background

Purpose:
To Design a database system that simulates how a trading platform like Robin hood manages user, accounts, stocks, watchlists, and transactions.

Problem:
Many students and beginner investors want to practice stock trading, but real trading apps require money and carry risk.

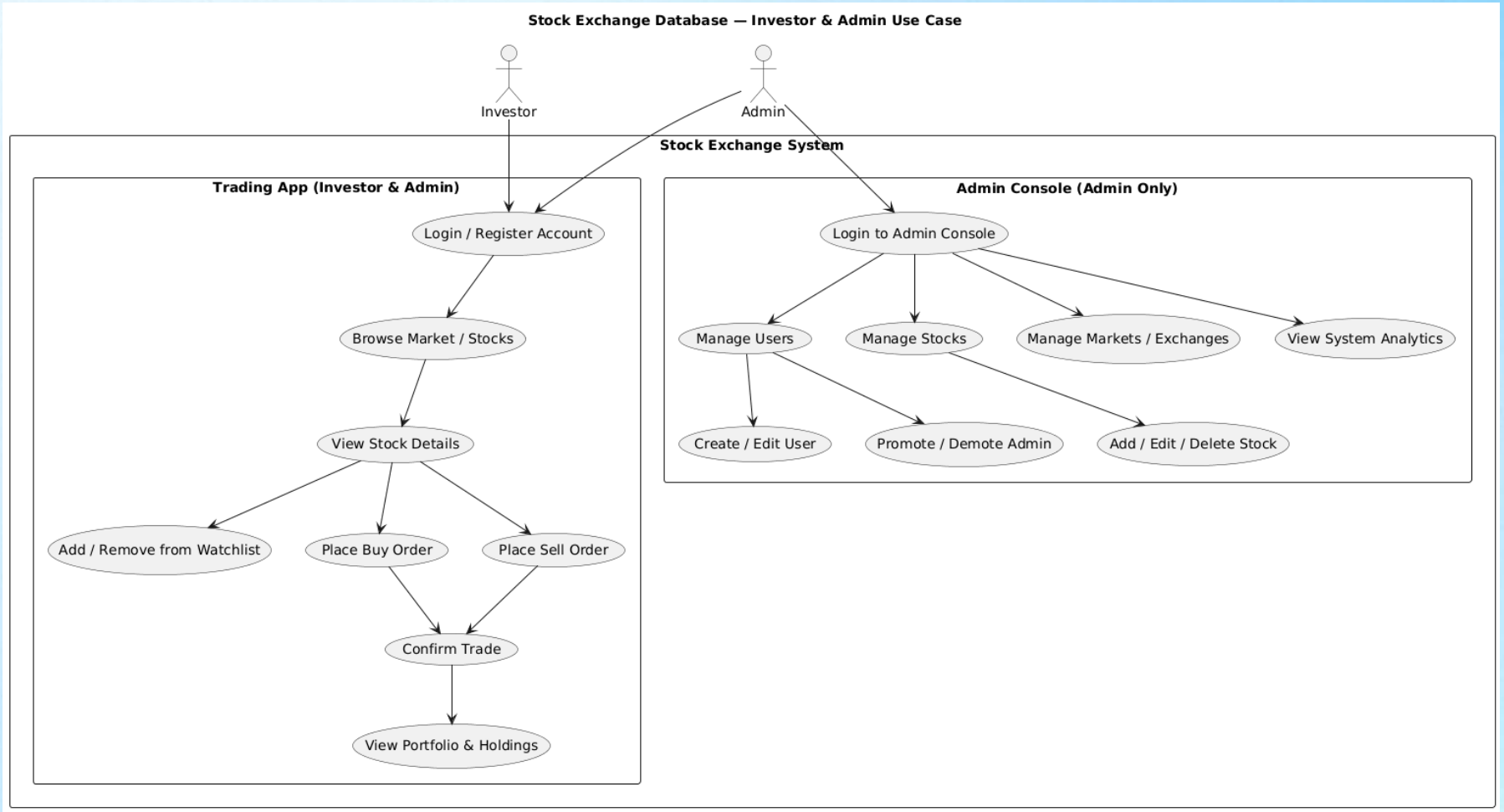
Solution:
Create a mock stock exchange database where users can register, view stock information, manage a virtual portfolio, and make simulated buy/sell transactions –
all working with our relational schema we made (users, accounts, stocks, orders holdings and watchlists).

Project Requirements/ Objectives

• Allow users to self-register, create accounts, and log in securely with role-based access (User and Admin).
• Store user information and virtual account balance.
• Display 145 + available stocks across 10 + sectors with name, symbol, sector, and exchange information.
• Enable users to add or remove stocks from their watchlist or remove stocks from watchlist individually.
• Let users buy and sell stocks using virtual currency.
• Record every transaction and order in the database with complete historical tracking.
• Show each user's portfolio and its current value.
• Maintain company and market information for each stock.
- Provide admin panel for user management, system-wide analytics, and stock database management.

-Goal*:* Design a normalized, query-efficient database to support all user interactions with role-based security and multi-account portfolio management.

Use Case Diagram- User (Investor) & Admin Roles



Software Stack

Software Stack

React.js

Frontend Framework

Python Flask

Backend Framework

MySQL

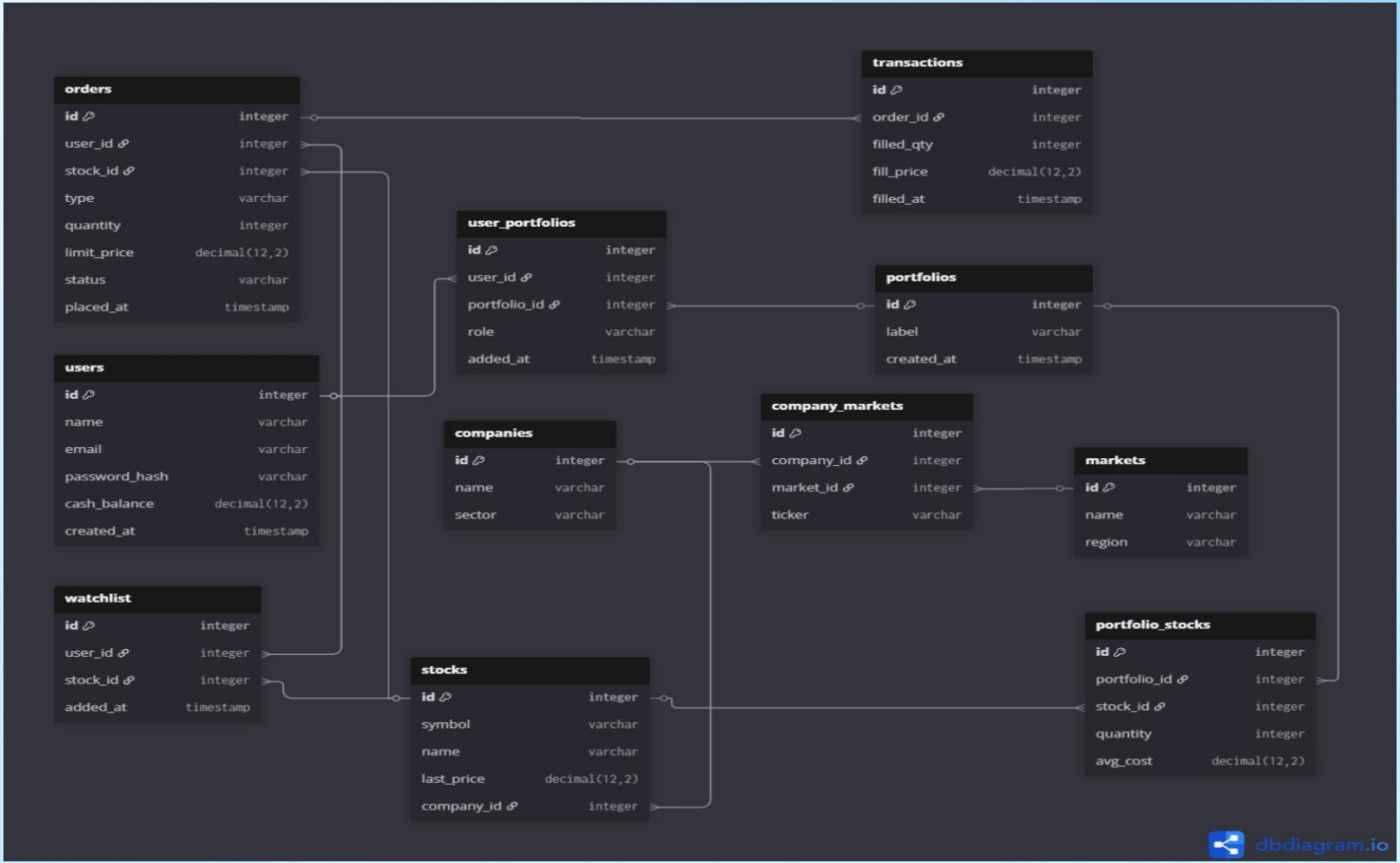
Relational Database

GitHub

Version Control / Collaboration

Layer	Technology	Purpose
Frontend	React.js, HTML, CSS	Builds user interface for login, trading, and portfolio pages.
Backend	Python (Flask)	Handles logic, routes, and API requests between UI and database.
Database	MySQL	Stores user, stock, transaction, and company data.
Tools	GitHub, VS Code, Draw.io	Used for development, version control, and diagrams.

E/R Diagram


orders
id integer
user_id integer
stock_id integer
type varchar
quantity integer
limit_price decimal(12,2)
status varchar
placed_at timestamp
transactions
id integer
order_id integer
filled_qty integer
fill_price decimal(12,2)
filled_at timestamp
user_portfolios
id integer
user_id integer
portfolio_id integer
role varchar
added_at timestamp
portfolios
id integer
label varchar
created_at timestamp
users
id integer
name varchar
email varchar
password_hash varchar
cash_balance decimal(12,2)
created_at timestamp
companies
id integer
name varchar
sector varchar
company_markets
id integer
company_id integer
market_id integer
ticker varchar
markets
id integer
name varchar
region varchar
watchlist
id integer
user_id integer
stock_id integer
added_at timestamp
stocks
id integer
symbol varchar
name varchar
last_price decimal(12,2)
company_id integer
portfolio_stocks
id integer
portfolio_id integer
stock_id integer
quantity integer
avg_cost decimal(12,2)
dbdiagram.io

Relation Specifications

User

PK user_id

- email
- role (USER / ADMIN)
- password_hash
- created_at

Account

PK account_id

FK user_id → User.user_id

- name
- cash_balance
- status (ACTIVE / FROZEN)
- opened_at

Security

PK security_id

- ticker (e.g., AAPL)
- name
- sector
- exchange

Watchlist

PK watchlist_id

FK user_id → User.user_id

- name
- created_at

Order

PK order_id

FK account_id → Account.account_id

FK security_id → Security.security_id

- side (BUY / SELL)
- type (MARKET / LIMIT)
- quantity
- limit_price (nullable)
- status (OPEN / FILLED / CANCELED)
- placed_at

Holding

PK holding_id

FK account_id → Account.account_id

FK security_id → Security.security_id

- quantity
- avg_cost
- updated_at

DailyPrice

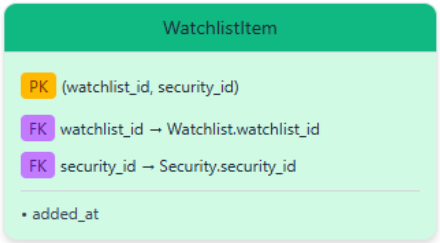
PK price_id

FK security_id → Security.security_id

- price_date
- open_price
- close_price
- high_price
- low_price
- volume

Relation Specifications

Many-to-Many Junction Tables



Schema models a real trading app:
users → accounts → orders/holdings, plus watchlists and price history.
Many-to-many: Watchlists and Securities are linked through WatchlistItem(watchlist_id, security_id) so one watchlist can have many stocks and each stock can appear in many watchlists.
One-to-many chains:
User → Accounts, User → Watchlists
Account → Orders, Account → Holdings
Security → Orders, Holdings, DailyPrices
Key constraints: unique emails and tickers, NOT NULL on core fields, and quantity > 0 for holdings/orders so data stays realistic.

Cascade deletes: removing a User, Account, Watchlist, or Security automatically cleans up related rows (orders, holdings, watchlist items, daily prices) so there are no orphan records.

Relationship Summary

9 Total Relationships

Many-to-Many (1):

→ WATCHLISTS ↔ SECURITIES (via WatchlistItem)

One-to-Many (8):

→ USERS → ACCOUNTS
→ USERS → WATCHLISTS
→ ACCOUNTS → ORDERS
→ ACCOUNTS → HOLDINGS
→ SECURITIES → ORDERS
→ SECURITIES → HOLDINGS
→ SECURITIES → DAILY_PRICES
→ WATCHLISTS → WATCHLIST_ITEMS

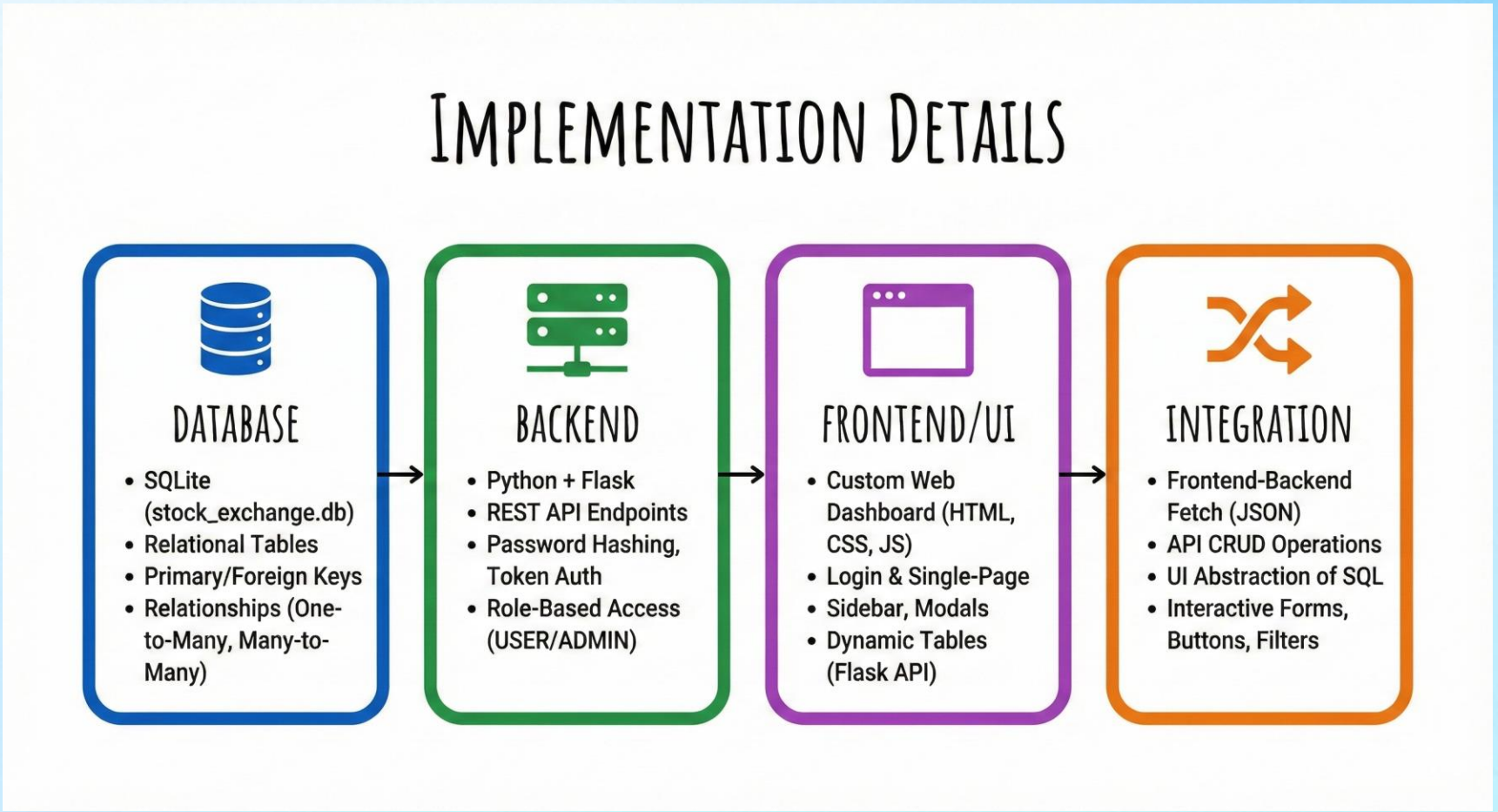
Key Constraints:

UNIQUE users.email, securities.ticker
NOT NULL All PKs/FKs, core fields
CHECK quantity > 0 for Holdings and Orders

Cascade Deletes:

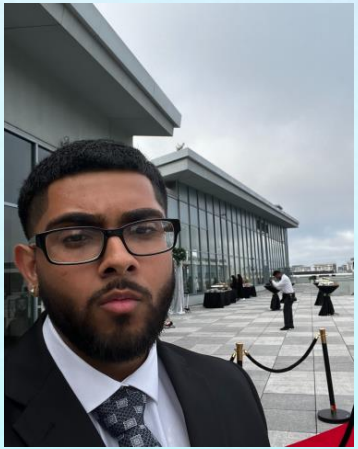
• USER → accounts, watchlists
• ACCOUNT → orders, holdings
• WATCHLIST → watchlist_items
• SECURITY → orders, holdings, daily_prices

Implementation details

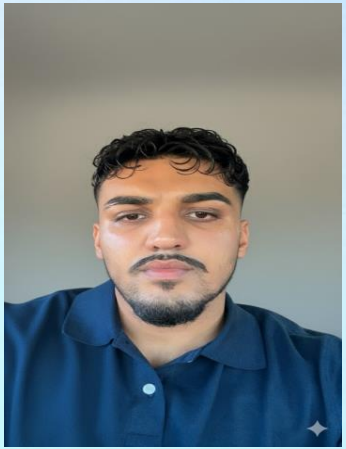


The Team

Ajay Grewal
Contact:
agrewal20@ucmerced.edu



Rohit Kumar
Contact:
rkumar43@ucmerced.edu



Arshdeep Dhaliwal
Contact:
adhaliwal25@ucmerced.edu

Thank you!!!


THE · UNIVERSITY · OF · CALIFORNIA · MERCED ·
LET THERE BE LIGHT
·2005·